

Calories Prediction — Data Analysis

English

This notebook covers the loading, preprocessing, and statistical analysis stages of the project, using the project's own modules — no duplicated logic:

- `src/data_loader.py` — loads the raw and processed datasets, reports basic dataset info, and exposes the project's file paths.
- `src/preprocessing.py` — the project's main preprocessing pipeline: column validation, type conversion, `Gender` normalization (including mapping short forms like `M / F`), target-aware missing-value handling, BMI calculation for missing values, physical-validity checks, and saving the processed dataset.
- `src/data_cleaning.py` — used here specifically for its generic `get_cleaning_report()` function, which compares any two DataFrames (before/after) and returns a before/after summary. It is reused to report on the output of `preprocessing.py`, even though the pipeline itself now comes from `preprocessing.py` rather than `data_cleaning.py`.
- `src/data_analysis.py` — every statistical check (basic info, missing values, duplicates, statistics, outliers, correlations) run both before and after preprocessing.

The flow is: **load raw data** → **analyze (raw)** → **preprocess & save** → **reload processed data** → **analyze (processed)**.

العربية

يغطي هذا الدفتر مراحل التحميل والمعالجة المسبقة والتحليل الإحصائي في المشروع، باستخدام وحدات المشروع نفسها — دون أي تكرار للمنطق:

- `src/data_loader.py` — لتحميل البيانات الخام والمعالجة، وعرض معلومات أساسية عنها، ومسارات ملفات المشروع.
- `src/preprocessing.py` — خط أنابيب المعالجة المسبقة الرئيسي للمشروع: التحقق من `M / F` بما في ذلك تحويل الاختصارات مثل `Gender` الأعمدة، تحويل الأنواع، توحيد قيم للقيم المفقودة، فحص الصحة BMI معالجة القيم المفقودة مع مراعاة المتغير المستهدف، حساب الفيزيائية للقيم، وحفظ البيانات المعالجة.
- `src/data_cleaning.py` — يُستخدم هنا فقط من أجل دالته العامة `get_cleaning_report()` والتي تقارن أي إطار بيانات (قبل/بعد) وترجع ملخصًا لما قبل، رغم أن خط الأنابيب `preprocessing.py` وبعد. تتم إعادة استخدامها لتوليد تقرير عن مخرجات `preprocessing.py` وليس `data_cleaning.py` نفسه أصبح من جميع الفحوصات الإحصائية (المعلومات الأساسية، القيم المفقودة، التكرارات، الإحصاءات، القيم المتطرفة، الارتباطات) تُنفَّذ قبل وبعد المعالجة المسبقة.

التسلسل هو: تحميل البيانات الخام ← تحليل (قبل) ← معالجة مسبقة وحفظ ← إعادة تحميل البيانات
المعالجة ← تحليل (بعد).

Project Setup & Imports

English

We add the project root to `sys.path` so the notebook can import directly from `src/`, the same way the modules import from each other.

العربية

`src/` ، حتى يتمكن الدفتر من الاستيراد مباشرة من مجلد `sys.path` نقوم بإضافة جذر المشروع إلى بنفس الطريقة التي تستورد بها الوحدات من بعضها البعض.

```
In [ ]: import sys
        from pathlib import Path

        import pandas as pd

        # -----
        # Make src/ importable, regardless of where the notebook is launched from
        # -----
        NOTEBOOK_DIR = Path.cwd()
        PROJECT_ROOT = NOTEBOOK_DIR.parent if NOTEBOOK_DIR.name == "notebooks" else NOTEBOOK_DIR.parent

        if str(PROJECT_ROOT) not in sys.path:
            sys.path.insert(0, str(PROJECT_ROOT))

        # -----
        # Project modules
        # -----
        from src.data_loader import (
            load_raw_data,
            load_processed_data,
            get_dataset_info,
            PROCESSED_DATA_FILE,
        )

        from src.preprocessing import (
            preprocess_data,
            save_processed_data,
        )

        from src.data_cleaning import get_cleaning_report

        from src.data_analysis import (
            generate_analysis_report,
            analyze_target,
            get_correlation,
        )
```

```
pd.set_option("display.max_columns", None)

print(f"Project root : {PROJECT_ROOT}")
print(f"Processed file: {PROCESSED_DATA_FILE}")
```

Project root : c:\Users\ayman\OneDrive\Desktop\calories-prediction

Processed file: C:\Users\ayman\OneDrive\Desktop\calories-prediction\data\processed\dataset_clean.csv

Load Raw Dataset

English

We load the raw dataset with `load_raw_data()` from `src/data_loader.py`, which resolves `data/raw/dataset.csv` relative to the project root, not the notebook's working directory.

العربية

والتى ، `src/data_loader.py` من `load_raw_data()` نقوم بتحميل البيانات الخام باستخدام `data/raw/dataset.csv` تحدد مسار بالنسبة لجذر المشروع، وليس مجلد عمل الدفتر

```
In [ ]: df_raw = load_raw_data()

info = get_dataset_info(df_raw)

for key, value in info.items():
    if key != "column_names":
        print(f"{key}: {value}")

print("\nColumns:")
for column in info["column_names"]:
    print(f"- {column}")

df_raw.head()
```

```
rows: 200
columns: 10
total_values: 2000
missing_values: 0
duplicate_rows: 0
```

Columns:

- Gender
- Age
- Height(cm)
- Weight(kg)
- BMI
- Running Time(min)
- Running Speed(km/h)
- Distance(km)
- Average Heart Rate
- Calories Burned

Out[]:

	Gender	Age	Height(cm)	Weight(kg)	BMI	Running Time(min)	Running Speed(km/h)	Distance(km)	Av
0	Male	49	184	74	20.4	78	9.5	3.00	
1	Female	56	182	70	23.2	98	10.2	5.23	
2	Male	49	170	85	27.4	91	10.4	7.61	
3	Male	21	181	59	24.0	89	13.4	14.19	
4	Male	47	172	86	29.8	79	9.8	3.84	

Analysis Before Preprocessing

English

`generate_analysis_report()` from `src/data_analysis.py` produces basic info, missing values, duplicates, statistics, and IQR-based outlier counts in a single call. We inspect the parts most relevant before preprocessing.

العربية

المعلومات الأساسية، `generate_analysis_report()` من `src/data_analysis.py` تُنتج في استدعاء واحد. نفحص IQR القيم المفقودة، التكرارات، الإحصاءات، وعدد القيم المتطرفة بطريقة الأجزاء الأكثر أهمية قبل المعالجة المسبقة.

```
In [ ]: raw_report = generate_analysis_report(df_raw)

print("Basic info:")
for key, value in raw_report["basic_info"].items():
    print(f" {key}: {value}")
```

```
print("\nMissing values (before preprocessing):")
print(raw_report["missing_values"])

print(f"\nDuplicate rows (before preprocessing): {raw_report['duplicates']}")
```

Basic info:

```
rows: 200
columns: 10
total_values: 2000
duplicate_rows: 0
missing_values: 0
```

Missing values (before preprocessing):

Series([], dtype: int64)

Duplicate rows (before preprocessing): 0

```
In [ ]: print("Outlier counts per numerical column (IQR method, before preprocessing):")
raw_report["outliers"]
```

Outlier counts per numerical column (IQR method, before preprocessing):

```
Out[ ]: Age                0
Height(cm)               0
Weight(kg)               0
BMI                      0
Running Time(min)        0
Running Speed(km/h)      0
Distance(km)             0
Average Heart Rate       0
Calories Burned          0
dtype: int64
```

```
In [ ]: raw_report["statistics"]
```

Out[]:

	count	mean	median	mode	min	max	range	std	va
Age	200	41.38500	44.000	50.00	18.0	64.00	46.00	13.225785	174.9
Height(cm)	200	175.09500	175.000	175.00	150.0	199.00	49.00	14.266170	203.5
Weight(kg)	200	75.18500	74.500	77.00	50.0	99.00	49.00	14.106585	198.9
BMI	200	24.24400	24.600	19.50	18.5	30.00	11.50	3.243558	10.5
Running Time(min)	200	74.34000	74.500	58.00	30.0	119.00	89.00	25.028916	626.4
Running Speed(km/h)	200	11.34150	11.350	11.20	7.0	15.00	8.00	2.167769	4.6
Distance(km)	200	8.92655	8.825	13.23	3.0	14.96	11.96	3.819304	14.5
Average Heart Rate	200	149.82000	150.500	139.00	121.0	179.00	58.00	16.949158	287.2
Calories Burned	200	584.08500	554.000	200.00	200.0	996.00	796.00	228.289659	52116.1



Preprocessing

English

We run `preprocess_data()` from `src/preprocessing.py` on the already-loaded `df_raw` (rather than `run_preprocessing_pipeline()`, which would reload the raw file again from disk). This pipeline validates columns, converts numeric types, normalizes `Gender` (including mapping `M / F / Man / Woman`), drops duplicates, fills missing values (rows with a missing target are dropped instead, since it cannot be safely inferred), calculates missing BMI values from height and weight, removes physically invalid values, sets final data types, and reorders columns.

We then save the result with `save_processed_data()`, and build a before/after report with `get_cleaning_report()` from `src/data_cleaning.py` — that function is generic, so it works on the output of `preprocessing.py` just as well as on `data_cleaning.py`'s own pipeline.

العربية

المُحمّلة `df_raw` على `src/preprocessing.py` من `preprocess_data()` نقوم بتشغيل التي كانت ستُعِيد تحميل الملف الخام من `run_preprocessing_pipeline()` بدلاً من) مسبقاً يقوم خط الأنابيب هذا بالتحقق من الأعمدة، تحويل الأنواع الرقمية، توحيد قيم (القرص من جديد حذف التكرارات، تعبئة القيم المفقودة (`M / F / Man / Woman` بما في ذلك تحويل `Gender` (باستثناء الصفوف التي يكون فيها المتغير المستهدف مفقودًا، حيث يتم حذفها لأنه لا يمكن استنتاجها

المفقودة من الطول والوزن، إزالة القيم غير الصالحة فيزيائياً، ضبط الأنواع BMI بأمان)، حساب قيم النهائية للبيانات، وإعادة ترتيب الأعمدة.

ونُنشئ تقرير قبل/بعد باستخدام `save_processed_data()` ، بعدها نحفظ النتيجة باستخدام `get_cleaning_report()` من `src/data_cleaning.py` — لذلك تعمل على `data_cleaning.py` تمامًا كما تعمل على خط الأنابيب الخاص بـ `preprocessing.py` مخرجات نفسه.

```
In [ ]: processed_df = preprocess_data(df_raw)

save_processed_data(processed_df)

=====
STARTING DATA PREPROCESSING
=====
Original shape: (200, 10)
Duplicates removed: 0
Invalid rows removed: 0
Final shape: (200, 10)
Remaining missing values: 0
=====
DATA PREPROCESSING COMPLETED
=====

Processed dataset saved to:
C:\Users\ayman\OneDrive\Desktop\calories-prediction\data\processed\dataset_clean.csv
```

```
In [ ]: cleaning_report = get_cleaning_report(df_raw, processed_df)

for key, value in cleaning_report.items():
    print(f"{key}: {value}")
```

```
original_rows: 200
cleaned_rows: 200
rows_removed: 0
original_columns: 10
cleaned_columns: 10
missing_values_before: 0
missing_values_after: 0
duplicates_before: 0
duplicates_after: 0
```

Load Processed Dataset

English

We reload the saved file with `load_processed_data()` rather than reusing `processed_df` in memory, to confirm the file on disk (the one every later stage of the project will actually read) matches what the pipeline produced.

العربية

بدلاً من الاعتماد على `load_processed_data()` نقوم بإعادة تحميل الملف المحفوظ باستخدام `processed_df` في الذاكرة، للتأكد من أن الملف الموجود على القرص (وهو ما ستقرأه فعلياً كل `processed_df` مرحلة لاحقة في المشروع) يطابق ما أنتجه خط الأنايب

```
In [ ]: df = load_processed_data()

print(f"Shape: {df.shape}")
df.head()
```

Shape: (200, 10)

```
Out [ ]:
```

	Gender	Age	Height(cm)	Weight(kg)	BMI	Running Time(min)	Running Speed(km/h)	Distance(km)
0	Male	49.0	184.0	74.0	20.4	78.0	9.5	3.00
1	Female	56.0	182.0	70.0	23.2	98.0	10.2	5.23
2	Male	49.0	170.0	85.0	27.4	91.0	10.4	7.61
3	Male	21.0	181.0	59.0	24.0	89.0	13.4	14.19
4	Male	47.0	172.0	86.0	29.8	79.0	9.8	3.84

Analysis After Preprocessing

English

We re-run `generate_analysis_report()` on the processed data to confirm missing values, duplicates, and outliers, then focus on the target variable and the correlation matrix using `analyze_target()` and `get_correlation()` from `src/data_analysis.py`.

العربية

على البيانات المعالجة للتأكد من القيم المفقودة `generate_analysis_report()` نعيد تشغيل والتكرارات والقيم المتطرفة، ثم نركز على المتغير المستهدف ومصفوفة الارتباط باستخدام `analyze_target()` و `get_correlation()` من `src/data_analysis.py`.

```
In [ ]: processed_report = generate_analysis_report(df)

print("Basic info:")
for key, value in processed_report["basic_info"].items():
    print(f" {key}: {value}")

print("\nMissing values (after preprocessing):")
print(processed_report["missing_values"])

print(f"\nDuplicate rows (after preprocessing): {processed_report['duplicates']}")
```


Basic info:
rows: 200
columns: 10
total_values: 2000
duplicate_rows: 0
missing_values: 0

Missing values (after preprocessing):
Series([], dtype: int64)

Duplicate rows (after preprocessing): 0

```
In [ ]: print("Outlier counts per numerical column (IQR method, after preprocessing):")
        processed_report["outliers"]
```

Outlier counts per numerical column (IQR method, after preprocessing):

Out[]: Age 0
Height(cm) 0
Weight(kg) 0
BMI 0
Running Time(min) 0
Running Speed(km/h) 0
Distance(km) 0
Average Heart Rate 0
Calories Burned 0
dtype: int64

```
In [ ]: processed_report["statistics"]
```

Out[]:

	count	mean	median	mode	min	max	range	std	var
Age	200	41.38500	44.000	50.00	18.0	64.00	46.00	13.225785	174.9
Height(cm)	200	175.09500	175.000	175.00	150.0	199.00	49.00	14.266170	203.5
Weight(kg)	200	75.18500	74.500	77.00	50.0	99.00	49.00	14.106585	198.9
BMI	200	24.24400	24.600	19.50	18.5	30.00	11.50	3.243558	10.5
Running Time(min)	200	74.34000	74.500	58.00	30.0	119.00	89.00	25.028916	626.4
Running Speed(km/h)	200	11.34150	11.350	11.20	7.0	15.00	8.00	2.167769	4.6
Distance(km)	200	8.92655	8.825	13.23	3.0	14.96	11.96	3.819304	14.5
Average Heart Rate	200	149.82000	150.500	139.00	121.0	179.00	58.00	16.949158	287.2
Calories Burned	200	584.08500	554.000	200.00	200.0	996.00	796.00	228.289659	52116.1



```
In [ ]: analyze_target(df, "Calories Burned")
```

```
Out[ ]: count      200.000000
mean      584.085000
median    554.000000
mode      200.000000
min       200.000000
max       996.000000
range     796.000000
std       228.289659
variance  52116.168618
Q1        395.750000
Q2        554.000000
Q3        791.500000
IQR       395.750000
dtype: float64
```

```
In [ ]: correlation_matrix = get_correlation(df)
correlation_matrix
```

Out[]:

	Age	Height(cm)	Weight(kg)	BMI	Running Time(min)	Running Speed(km/h)	Dist
Age	1.000000	-0.115622	-0.062736	-0.036499	0.008984	-0.113733	
Height(cm)	-0.115622	1.000000	-0.030276	-0.115931	0.112313	-0.028401	
Weight(kg)	-0.062736	-0.030276	1.000000	-0.050567	-0.109841	0.061732	
BMI	-0.036499	-0.115931	-0.050567	1.000000	-0.039732	-0.106841	
Running Time(min)	0.008984	0.112313	-0.109841	-0.039732	1.000000	-0.061102	
Running Speed(km/h)	-0.113733	-0.028401	0.061732	-0.106841	-0.061102	1.000000	
Distance(km)	-0.020051	0.067200	0.018295	0.144888	-0.056769	0.008205	
Average Heart Rate	-0.115899	0.062085	0.017206	-0.046920	-0.035617	-0.010710	
Calories Burned	0.035348	0.100808	0.070512	-0.059128	-0.056699	0.013207	

```
In [ ]: target_correlations = (
    correlation_matrix["Calories Burned"]
    .drop("Calories Burned")
)

correlation_table = pd.DataFrame({
    "Feature": target_correlations.index,
    "Correlation": target_correlations.values,
    "Absolute Correlation": target_correlations.abs().values,
}).sort_values("Absolute Correlation", ascending=False).reset_index(drop=True)
```

correlation_table

Out[]:

	Feature	Correlation	Absolute Correlation
0	Height(cm)	0.100808	0.100808
1	Weight(kg)	0.070512	0.070512
2	BMI	-0.059128	0.059128
3	Running Time(min)	-0.056699	0.056699
4	Distance(km)	-0.051363	0.051363
5	Age	0.035348	0.035348
6	Average Heart Rate	0.014701	0.014701
7	Running Speed(km/h)	0.013207	0.013207

Summary

English

- **Loading:** `src/data_loader.py` resolved the raw and processed dataset paths from the project root.
- **Preprocessing:** `src/preprocessing.py`'s `preprocess_data()` handled column validation, type conversion, `Gender` normalization, target-aware missing-value handling, BMI calculation, and physical-validity checks, then `save_processed_data()` wrote the result to `data/processed/dataset_clean.csv`.
- **Reporting:** `get_cleaning_report()` from `src/data_cleaning.py` was reused (unmodified) to summarize the before/after effect of the `preprocessing.py` pipeline.
- **Analysis:** `src/data_analysis.py` produced the full statistical picture both before and after preprocessing — basic info, missing values, duplicates, statistics, IQR-based outliers, target analysis, and the correlation matrix.
- **Next step:** the sorted correlation table above is the starting point for feature selection, and the processed CSV is ready for the visualization notebook.

العربية

- مسارات البيانات الخام والمعالجة انطلاقًا من جذر `src/data_loader.py` التحميل: حدّدت المشروع.
- التحقق من `src/preprocessing.py` من `preprocess_data()` المعالجة المسبقة: تولّدت معالجة القيم المفقودة مع مراعاة المتغير، `Gender` الأعمدة، تحويل الأنواع، توحيد قيم `save_processed_data()` وفحص الصحة الفيزيائية للقيم، ثم كتبت BMI المستهدف، حساب النتيجة في `data/processed/dataset_clean.csv`.

- `src/data_cleaning.py` من `get_cleaning_report()` التقرير: تمت إعادة استخدام `preprocessing.py` لتلخيص أثر خط أنابيب (دون تعديل) قبل وبعد.
- الصورة الإحصائية الكاملة قبل وبعد المعالجة `src/data_analysis.py` التحليل: أنتجت المسبقة — المعلومات الأساسية، القيم المفقودة، التكرارات، الإحصاءات، القيم المتطرفة بطريقة تحليل المتغير المستهدف، ومصفوفة الارتباط، IQR.
- CSV الخطوة التالية: جدول الارتباط المرتب أعلاه هو نقطة الانطلاق لاختيار الخصائص، وملف المعالج جاهز الآن لدقتر التصور البصري.